Python Fundamentals

Complete Beginner Study Guide

Variables · Data Types · Lists · Tuples · Dictionaries · Sets Conditions · Loops · Functions

Based on freeCodeCamp Full Python Course for Beginners

TABLE OF CONTENTS

CHAPTER 1

Variables

Naming and storing data in Python

A **variable** is a named container that stores a value. Think of it as a labeled box — you put something inside the box and refer to it by the label whenever you need it.

Syntax

```
# variable_name = value
name = 'Chardon' # stores a string
age = 38 # stores an integer
salary = 75000.50 # stores a decimal
is_veteran = True # stores a boolean

# Print values
print(name) # Output: Chardon
print(age) # Output: 38
```

Naming Rules

- Must start with a letter or underscore (_), not a number
- Can only contain letters, numbers, and underscores
- Case-sensitive: name and Name are two different variables
- Cannot use Python reserved words (if, for, while, def, class...)

```
# Good variable names
user_name = 'Chardon'
monthly_income = 3831.30
_private_var = 42

# Bad names (will cause errors)
# 2fast = 'nope' # starts with number
# my-var = 'nope' # hyphens not allowed
# for = 'nope' # reserved keyword
```

AI Automation Tip

In AI automation scripts, you'll use variables constantly to store API responses, prompt text, model names, and output results. Every automation workflow you build starts with well-named variables.

CHAPTER 2

Data Types

The different kinds of data Python can hold

Every value in Python has a **data type** that tells Python what kind of data it is and what you can do with it. The four core types you must know are: **String, Integer, Float,** and **Boolean**.

Data Type	What It Stores	Example	Use Case
str (String)	Text — anything in quotes	'Hello World'	Names, messages, prompts
int (Integer)	Whole numbers	42	Counts, ages, indexes
float (Float)	Decimal numbers	3.14	Prices, percentages, scores
bool (Boolean)	True or False only	True / False	On/off switches, conditions

Code Examples

```
# Strings
first_name = 'Chardon'
greeting = "Hello, World!"
multi = 'Python is great for AI'

# Integers
year = 2025
num_channels = 2
subscribers = 10000

# Floats
va_compensation = 3831.30
rpm_rate = 12.50

# Booleans
is_monetized = True
is_employed = False

# Check the type of any variable
print(type(first_name)) #
print(type(year)) #
```

```
print(type(va_compensation)) #
print(type(is_monetized)) #
```

Type Conversion

You can convert between types using built-in functions:

```
age_string = '38' # This is a string
age_int = int(age_string) # Convert to integer: 38
age_float = float(age_string) # Convert to float: 38.0
back_to_str = str(age_int) # Convert back to string: '38'
```

Why This Matters for AI

When you call an AI API and get a response, it comes back as a string. You'll often need to convert parts of that response to integers or floats to do calculations. Type conversion is used in almost every automation script.

CHAPTER 3

Lists

Ordered, changeable collections of items

A **list** stores multiple items in a single variable, in a specific order. Lists use square brackets [ ] and items can be changed after creation — this is called being **mutable**.

```
# Creating lists
fruits = ['apple', 'banana', 'cherry']
numbers = [10, 20, 30, 40, 50]
mixed = ['Chardon', 38, True, 3.14] # can mix types

# Accessing items by INDEX (starts at 0!)
print(fruits[0]) # apple (first item)
print(fruits[1]) # banana (second item)
print(fruits[-1]) # cherry (last item)

# Changing items
fruits[1] = 'blueberry'
print(fruits) # ['apple', 'blueberry', 'cherry']
```

Essential List Methods

```
channels = ['Money Grizzly', 'AI News']

channels.append('Finance Tips') # add to end
channels.insert(0, 'New Channel') # insert at position 0
channels.remove('AI News') # remove by value
channels.pop() # remove last item
channels.sort() # sort alphabetically
print(len(channels)) # get length/count

# Slicing — get a portion of a list
items = [1, 2, 3, 4, 5]
print(items[1:3]) # [2, 3] (index 1 up to but not including 3)
print(items[:3]) # [1, 2, 3] (from start)
print(items[2:]) # [3, 4, 5] (to end)
```

Real-World Use

In AI automation, lists are used everywhere: storing a list of prompts to process, collecting API responses, holding file names to iterate through, or storing the results from an automated workflow before saving them.

CHAPTER 4

Tuples

Ordered, unchangeable collections

A **tuple** is like a list, but it's **immutable** — once created, you cannot change it. Tuples use parentheses ( ). Use tuples for data that should never change: coordinates, RGB colors, fixed configurations.

```
# Creating tuples
coordinates = (40.7128, -74.0060) # New York latitude/longitude
rgb_color = (255, 165, 0) # Orange in RGB
days = ('Mon', 'Tue', 'Wed', 'Thu', 'Fri')

# Accessing — same as lists
print(coordinates[0]) # 40.7128
print(days[-1]) # Fri

# Tuples are IMMUTABLE — this causes an error:
# coordinates[0] = 35 # TypeError: 'tuple' object does not support item assignment

# Tuple unpacking — assign multiple variables at once
lat, lon = coordinates
print(lat) # 40.7128
print(lon) # -74.0060
```

List vs Tuple — Quick Comparison

Feature	List [ ]	Tuple ( )
Changeable?	Yes (mutable)	No (immutable)
Speed	Slightly slower	Faster
Use when...	Data will change	Data is fixed/constant
Syntax	[1, 2, 3]	(1, 2, 3)

CHAPTER 5

Dictionaries

Key-value pairs — Python's most powerful data structure

A **dictionary** stores data as **key: value** pairs, like a real dictionary where you look up a word (key) to find its definition (value). Dictionaries use curly braces { }. This is the structure you'll use most in AI work — every API response comes back as a dictionary!

```
# Creating a dictionary
veteran = {
'name': 'Chardon Phillips',
'branch': 'Air Force',
'years_served': 8,
'disability_rating': 100,
'is_veteran': True
}

# Accessing values by key
print(veteran['name']) # Chardon Phillips
print(veteran['disability_rating']) # 100

# Safe access — won't crash if key missing
print(veteran.get('rank', 'Unknown')) # Unknown

# Adding / updating
veteran['city'] = 'Las Vegas' # add new key
veteran['years_served'] = 9 # update existing

# Removing
del veteran['is_veteran']

# Get all keys, values, or pairs
print(veteran.keys()) # dict_keys(['name', 'branch', ...])
print(veteran.values()) # dict_values(['Chardon Phillips', ...])
print(veteran.items()) # dict_items([('name', 'Chardon'), ...])
```

Critical for AI Automation

Every response from an AI API (Claude, OpenAI, etc.) is a dictionary. When you call an API, you'll write code like: response['content'][0]['text'] to extract the AI's reply. Mastering dictionaries is non-negotiable for AI work.

CHAPTER 6

Sets

Unique, unordered collections — no duplicates allowed

A **set** stores unique items — it automatically removes duplicates. Sets use curly braces { } but unlike dictionaries, they don't have keys. Sets are unordered, meaning you can't access items by index.

```
# Creating a set
skills = {'Python', 'SQL', 'Networking', 'Python'} # duplicate removed!
print(skills) # {'Python', 'SQL', 'Networking'} — no duplicate

# Creating from a list (easy way to remove duplicates)
tags = ['AI', 'Finance', 'AI', 'Python', 'Finance']
unique_tags = set(tags)
print(unique_tags) # {'AI', 'Finance', 'Python'}

# Adding and removing
skills.add('AI Automation')
skills.remove('SQL')
skills.discard('NoSQL') # won't crash if not found

# Set operations (think Venn diagrams)
set_a = {'Python', 'SQL', 'Excel'}
set_b = {'Python', 'AI', 'Excel'}

print(set_a & set_b) # intersection: {'Python', 'Excel'}
print(set_a | set_b) # union: {'Python', 'SQL', 'Excel', 'AI'}
print(set_a - set_b) # difference: {'SQL'}
```

When to Use Sets

Use sets when you need to: (1) remove duplicates from a list, (2) check if an item exists quickly, or (3) compare two collections to find common or unique items. In automation, this is useful for deduplicating API results or tracking which items have already been processed.

CHAPTER 7

Conditions

Making decisions with if / elif / else

Conditions let your program make **decisions**. Python uses **if**, **elif** (else if), and **else** to branch code based on whether something is True or False.

```
# Basic if statement
age = 38
if age >= 18:
print('Adult') # indented 4 spaces — REQUIRED in Python

# if / else
score = 85
if score >= 60:
print('Pass')
else:
print('Fail')

# if / elif / else (multiple conditions)
rating = 100
if rating == 100:
print('100% service connected')
elif rating >= 70:
print('High disability rating')
elif rating >= 30:
print('Moderate disability rating')
else:
print('Below 30%')
```

Comparison Operators

```
# == equal to 5 == 5 → True
# != not equal to 5 != 3 → True
# > greater than 10 > 5 → True
```

```
# < less than 3 < 7 → True
# >= greater or equal 5 >= 5 → True
# <= less or equal 4 <= 6 → True

# Logical operators
# and — both must be True
# or — at least one must be True
# not — reverses True/False

is_veteran = True
disability_pct = 100
if is_veteran and disability_pct == 100:
print('Eligible for full VR&E; benefits')
```

In Automation Scripts

Conditions are the brain of every automation. You'll write: 'if the API call succeeded, process the response. elif there was a rate limit, wait and retry. else, log the error.' Every agentic AI workflow runs on if/elif/else logic.

CHAPTER 8

Loops

Repeating tasks with for and while

Loops let you repeat a block of code multiple times without writing it over and over. Python has two loop types: **for** (loop a set number of times) and **while** (loop until a condition is False).

The FOR Loop

```
# Loop through a list
channels = ['Money Grizzly', 'AI Finance', 'Tech Basics']
for channel in channels:
print(f'Channel: {channel}')

# Loop a set number of times with range()
for i in range(5): # 0, 1, 2, 3, 4
print(i)

for i in range(1, 6): # 1, 2, 3, 4, 5
print(i)

# Loop through a dictionary
veteran = {'name': 'Chardon', 'rating': 100}
for key, value in veteran.items():
print(f'{key}: {value}')
```

The WHILE Loop

```
# Keep looping while condition is True
count = 0
while count < 5:
print(f'Count: {count}')
count += 1 # IMPORTANT: must update variable or infinite loop!

# break — exit loop early
for num in range(10):
if num == 5:
```

```
    break # stop at 5
    print(num)

    # continue — skip current iteration
    for num in range(10):
    if num % 2 == 0: # if even
    continue # skip evens
    print(num) # only prints odd numbers
```

Loops in AI Automation

You'll use for loops constantly: looping through a list of prompts, processing multiple files, iterating over API results, or sending batch requests. While loops handle retry logic — 'keep trying until the API responds successfully.'

CHAPTER 9

Functions

Reusable blocks of code — the backbone of automation

A **function** is a named block of code you define once and can call (use) as many times as you need. Functions keep your code organized, avoid repetition (DRY: Don't Repeat Yourself), and make automation scripts readable and maintainable.

```
# Define a function with 'def'
def greet(): # no parameters
print('Hello, Chardon!')

greet() # Call the function — Output: Hello, Chardon!

# Function with parameters (inputs)
def greet_user(name):
print(f'Hello, {name}!')

greet_user('Chardon') # Hello, Chardon!
greet_user('Eddie') # Hello, Eddie!

# Function with return value (output)
def calculate_monthly_income(subscribers, rpm):
monthly_views = subscribers * 0.1 # estimate
income = (monthly_views / 1000) * rpm
return income

result = calculate_monthly_income(10000, 12)
print(f'Estimated monthly income: ${result}') # $12.0
```

Default Parameters & Multiple Returns

```
# Default parameter values
def create_channel(name, niche='Finance', language='English'):
return f'{name} | Niche: {niche} | Lang: {language}'

print(create_channel('Money Grizzly')) # uses defaults
print(create_channel('AI Bot', niche='Tech')) # overrides niche only
```

```
# Return multiple values (comes back as a tuple)
def get_stats(views, rpm=12):
monthly_ad = (views / 1000) * rpm
annual_ad = monthly_ad * 12
return monthly_ad, annual_ad

monthly, annual = get_stats(150000)
print(f'Monthly: ${monthly}, Annual: ${annual}')
```

Functions Are Everything in Automation

Every AI automation workflow is a collection of functions: one function to call the API, one to parse the response, one to save to a file, one to send a notification. You define each function once, then chain them together to build powerful automated systems.

CHAPTER 10

Quick Reference Cheat Sheet

All fundamentals at a glance

CONCEPT	SYNTAX / EXAMPLE	KEY POINT
Variable	`name = 'Chardon'`	Named container for data
String	`'Hello' or "Hello"`	Text in quotes
Integer	`age = 38`	Whole number
Float	`price = 9.99`	Decimal number
Boolean	`is_active = True`	Only True or False
List	`[1, 2, 3]`	Ordered, changeable
List access	`my_list[0]`	Index starts at 0
List add	`my_list.append(4)`	Add to end
Tuple	`(1, 2, 3)`	Ordered, UNCHANGEABLE
Tuple unpack	`a, b = (1, 2)`	Assign multiple vars
Dictionary	`{'key': 'value'}`	Key-value pairs
Dict access	`d['key'] or d.get('key')`	Access by key name
Dict add	`d['new'] = 'value'`	New key = new value
Set	`{1, 2, 3}`	Unique items, no order
Set from list	`set([1,1,2,3])`	Removes duplicates
If / elif / else	`if x > 5:`	Decision branching
Comparison ops	`==  !=  >  <  >=  <=`	Compare values
Logical ops	`and  or  not`	Combine conditions
For loop	`for x in my_list:`	Loop through items
Range	`range(1, 6)`	Numbers 1,2,3,4,5
While loop	`while count < 10:`	Loop while True
Break	`break`	Exit loop early
Continue	`continue`	Skip to next iteration
Define function	`def my_func(param):`	Create reusable block
Return value	`return result`	Send value back out
Default param	`def f(x, y=10):`	y is optional input
f-string	`f'Hello {name}'`	Insert variable in string
Type check	`type(variable)`	Find data type

Type convert	`int('5')  str(42)`	Change data type